

vector simulation becomes impractical on anything. That ceiling is precisely why quantum hardware is interesting, and in the meantime, it means your development loop runs locally and free.

Actually running something

Enough of architecture. Here is the shortest useful path to run a quantum program.

Install Qiskit and the Aer simulator, build a Bell state, and run it locally. Two entangled qubits, the traditional first program.

```
python
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit import transpile

qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

sim = AerSimulator()
result = sim.run(transpile(qc, sim),
shots=1024).result()
print(result.get_counts())
```

You should see roughly 500 counts of 00 and 500 of 11, with nothing in between. That absence of 01 and 10 is the entanglement. Run it on real hardware later and you will see a few hundred stray counts in those buckets, which is the noise made visible.

Do not start on real hardware. I mean that as practical advice rather than gatekeeping. Simulator first, always, because a circuit that is wrong on a simulator will be wrong and slow, and possibly billed on a QPU.

When you are ready for hardware, the pattern is the same across providers: authenticate, pick a backend, transpile against that backend's real coupling map and gate set, submit, wait, collect. The transpile step against the actual device is the part people skip and then wonder why their results are garbage.

On cost, the model varies but the shape is consistent. Braket charges per task plus per shot. IBM offers open access with monthly time limits on real devices and paid plans beyond that. D-Wave gives free metered time. Simulators are cheap or free everywhere.

The failure mode to avoid is a loop that submits jobs to hardware while you debug. Rate limits and budget alarms exist for the same reason they exist in any cloud service, and quantum jobs are billed in units that are unfamiliar enough that people misjudge them. Set a spending cap before your first hardware submission, not after.

What to actually do with quantum computing

My honest read, and clients do not always enjoy hearing it.

For the overwhelming majority of organisations, the correct engagement level right now is one or two engineers spending a few hours a month learning, plus standing watch on developments in your specific industry.

The exceptions are real but narrow — pharmaceutical and materials work, where molecular simulation is the most credible near-term application, and certain logistics and portfolio optimisation problems, though check the classical baseline first and expect to be disappointed. Cryptography, which is a different conversation entirely and the one that actually has a deadline.

That last one deserves a sentence on its own. Post-quantum cryptography is not speculative. NIST standards are published, migration guidance exists, and the harvest-now-decrypt-later risk means data you transmit today with classical encryption may be readable

later. If your organisation handles anything with a long confidentiality horizon, post-quantum cryptography migration is a present-tense infrastructure project, and it has nothing to do with whether you ever run a quantum circuit.

I find that distinction gets lost constantly. Quantum computing is a research capability you are watching. Post-quantum cryptography is work you should have already started.

Where this leaves us

The agricultural equipment company did not buy a quantum computer. Two of their engineers now have IBM Quantum accounts and one of them has become genuinely good at this, which is a far better outcome for roughly zero rupees.

That is the shape of the opportunity in 2026. The hardware is remarkable, the results are not yet commercially useful, and the barrier to learning has collapsed to nearly nothing because the entire software layer is open source and the simulators run on your laptop. You can spend a weekend getting properly oriented for free.

The organisations that will benefit when the hardware matures are not the ones buying access now. They are the ones that will have people who understand the problem shapes, know what a variational algorithm is, and can tell a real advantage claim from a marketing one. That is a training question, and training takes years, which is the honest argument for starting now rather than waiting.

Install Qiskit. Build the Bell state. Read the calibration data for a real device and sit with what the error rates mean. That is a genuinely useful afternoon, and it costs you nothing. **END** 🐧

By: Vishal Tanna

The author is the CEO of TechCrave, a technology services company that helps enterprises adopt open source, cloud native, and AI-driven solutions. He has over 15 years of experience in software development and technology leadership, and is particularly interested in scalable architectures and digital transformation.